## 1. Diseño del Esquema de Sincronización

Para manejar los recursos del juego sin problemas, usamos hilos independientes para el Puma, los Perros y el Juez de cada mesa. Para proteger el tablero y evitar que dos fichas se muevan al mismo tiempo a la misma casilla, creamos un candado principal llamado 'mutex_tablero'.

Antes de moverse, cualquier ficha tiene que pedir este candado. Una vez que lo tiene, revisa si su movimiento es válido, actualiza el tablero y luego suelta el candado rápidamente. También agregamos un 'mutex_consola' independiente para que, al imprimir el tablero en la pantalla, los textos de las distintas mesas no se mezclen.

## 2. Análisis de Riesgos de Concurrencia

Durante el desarrollo identificamos y solucionamos los siguientes problemas:

- Deadlock (Interbloqueo): Había un riesgo de que el programa se quedara congelado si un hilo tenía el candado del tablero y pedía el de la consola al mismo tiempo. Lo arreglamos separando las tareas: ahora el Juez bloquea el tablero, hace una copia rápida de las posiciones, suelta el candado del tablero y recién ahí pide el candado de la consola para imprimir la copia. Nunca se sostienen dos candados a la vez.
- Condiciones de Carrera (Race Conditions): Si dos fichas quieren moverse al mismo lugar exacto a la vez, no hay problema porque la validación y el movimiento se hacen de forma atómica dentro del 'mutex_tablero'. El primero que llega toma el lugar y el segundo se rechaza.
- Inanición (Starvation): Como hay 12 perros y solo 1 puma, los perros podrían acaparar el acceso al tablero. Sin embargo, como el bloqueo de la matriz dura apenas microsegundos y cada ficha 'duerme' un tiempo aleatorio antes de volver a intentar moverse, el sistema operativo logra repartir los turnos a todos por igual, evitando que alguna pieza se quede sin jugar.

## 3. Evaluación

Decidimos usar un solo candado (mutex) para todo el tablero en lugar de poner un candado por cada una de las casillas.

Si hubiéramos puesto un candado por casilla, las fichas podrían haberse movido al mismo tiempo en distintas partes del tablero, pero el código habría sido mucho más complejo y propenso a errores graves (como deadlocks al intentar bloquear varias casillas a la vez durante el salto del Puma).

Como actualizar una matriz pequeña de 5x5 toma un tiempo casi invisible para el procesador comparado con lo que tardan las fichas en dormir entre cada turno, el rendimiento general del juego no se ve afectado. Esta decisión hace que el código sea mucho

más seguro, estable, fácil de entender y libre de congelamientos, cumpliendo perfectamente con los requerimientos.